

Deep Q-Networks and MCTS to find an Optimal Policy for Secret Hitler

Shaan Narayan

*Ann and H.J. Smead Aerospace
Engineering Sciences
University of Colorado Boulder
shna4849@colorado.edu*

Adam Yarrow

*Ann and H.J. Smead Aerospace
Engineering Sciences
University of Colorado Boulder
adne5155@colorado.edu*

Pedro Lemos

*Department of Computer Science
University of Colorado Boulder
peal3949@colorado.edu*

Abstract—This project represents the board game *Secret Hitler* as a simplified MDP and implements DQN and MCTS to find an optimal policy for the fascist team. *Secret Hitler* makes for an interesting decision making under uncertainty problem as both the roles of the players, and the optimal action to take, are uncertain depending on the role of the agent in the game. It was found that for cases where the election of the chancellor is highly dependent on the suspicion of every player being fascist, DQN and MCTS converged on an optimal policy that outperformed the baseline greedy and random policies. However, the differences between the optimal DQN and MCTS policies were small relative to the greedy policy for all cases due to simplified modeling assumptions. Overall the solution process demonstrated that DQN or MCTS effectively solved this simplified model of *Secret Hitler* and likely could be used to solve a more advanced model.

I. INTRODUCTION

Secret Hitler is a hidden role, social deception board game where players are secretly divided into two teams, Liberals and Fascists, with one of the Fascists taking on the role of Hitler. Each round the players elect a government in the form of a president and chancellor (the president is chosen through clockwise rotation and chancellor is selected by voting among the players). The government then can pass either a Fascist or Liberal policy based on both a random draw of cards from the deck, and their individual motives/roles. The Liberal team wins if five Liberal policies are passed and the Fascist team wins if six Fascist policies are passed (there are some other game mechanics that are not considered in this paper, the full rules can be found at www.secrethitler.com). The Liberal players and Hitler do not know the roles of any players, whereas the Fascist players have full observability of all player's true roles.

This game is interesting from a decision making under uncertainty perspective for two main reasons. Firstly, for the liberal players there is uncertainty about what roles players have. This uncertainty manifests in the voting process where the liberal players (who hold the majority) will opt to elect chancellors who behave in a liberal manner to ensure that liberal policies are played. However, this allows fascist players to exploit this uncertainty in roles and get into positions of power. Secondly, for the fascist players there is uncertainty associated with what policy to play/discard when in positions of power (chancellor or president roles). As a fascist player, to win the game you need to appear as a liberal so that you can

become the chancellor and force fascist policies to be played. To do this, fascist players may initially need to play liberal policies, which is in direct opposition to their primary goal, hence introducing uncertainty in the actions they take.

In our implementation of a simplified version of this game we chose to focus on playing from the fascists' perspective. This was done because a liberal player has almost no reason to discard a liberal policy, but a fascist has to decide between playing a fascist policy to further their progress towards winning or playing a liberal policy to avoid suspicion. We also further simplified the action space by incorporating the voting mechanism into the generative model as a random process parameterized by the suspicion level of every player. We then solved this MDP (Markov Decision Process) to find the policy for the fascist player to take using both a Deep Q-Learning Network (DQN) and Monte Carlo Tree Search (MCTS), as the game involves a continuous state space and is modeled through a generative model rather than formal state transition matrices. We compare both of these solvers to the performance of a random heuristic policy and a greedy policy for evaluation.

II. BACKGROUND AND RELATED WORK

A. *Secret Hitler* and other Social Deception Games Problem Formulation

This project was strongly influenced by the work of Zhang and Peng [1] who modeled *Secret Hitler* as an MDP. In their paper they also assumed the fascists' role (not Hitler) within a five player game and set out to find the optimal policy for the fascist's to win the game. Zhang and Peng modeled all other agents with fixed heuristic policies and incorporated a suspicion level of each player which influenced the reward returned at the end of each round. This paper simplifies voting mechanics by randomly selecting the chancellor and having suspicion levels affect the reward function rather than state transitions.

To model how the suspicion level of each player was updated between game rounds Aitchison, Benke and Sweetser [2] introduced a model of deception called Bayesian belief manipulation. In this model, the probability that a player is a certain role (e.g. fascist) is updated through a Bayesian model

based on the action taken, the prior probability about that players role, and the probabilities that a player would take an action given that they are a certain role (in the paper it is assumed that players know the policies of each role).

Reinhardt [3] also attempted to find an optimum policy for Secret Hitler, focusing on modeling the game as a POMDP and using Single Observer Information Set Monte Carlo Tree Search, rather than Q-learning as found in [1]. However, the paper experienced problems modeling the finite deck of policy cards and computational issues arising from a large and complex state-space.

B. Deep Q-Learning for Solving MDPs

Deep Q-Networks (DQN) were first introduced by Mnih et al. as an extension of Q-Learning, where a neural network is trained to approximate the optimal state-action value function $Q^*(s,a)$ of an environment [4]. These networks have had great success solving Markov Decision Processes such as Atari Games.

To improve upon the standard DQN algorithm, Hessel et al. examined the combination of six DQN improvements (Rainbow DQN) and observed faster training and better performance compared to regular DQN [5]. These improvements included double Q-learning, prioritized replay, dueling networks, multi-step learning, distributional reinforcement learning, and noisy nets.

Furthermore, Sutton et al. propose using a Semi-Markov Decision Process framework for doing reinforcement learning on problems where actions take variable time to complete [6]. In this framework, rewards and state transitions of an MDP are accumulated between an agent's decision points (meaningful actions), which can be useful for formulations where an agent doesn't always have an impactful action at each time step.

In addition to these improvements, Ali and Tirlé demonstrated the effectiveness of action masking within the DQN framework, allowing the network to learn faster in environments with infeasible or illegal state-action pairs [7]. This was accomplished by setting $Q(s,a)$ to $-\infty$ for illegal state-action pairs, greatly accelerating learning compared to waiting for the agent to learn which actions were illegal.

C. MCTS for Solving Continuous State MDPs

Monte Carlo Tree Search (MCTS) is online algorithm that builds a search tree by simulating trajectories from the current state. Online algorithms are adaptive and naturally important in the context of Secret Hitler for two reasons. Firstly, they allow for the game to be played out in real time against real agents. Secondly, owing to the large state space of the game it is intractable to solve it using exact methods such as value iteration, hence using an adaptive online solution helps make the solution space computationally realizable.

Starting from the current decision state, MCTS builds a search tree using an exploration vs. exploitation rule (commonly

the Upper Confidence Bound) and stores the visit counts, transition counts, and Q-value estimates for each node. After enough iterations, the algorithm chooses the action that yields the highest expected utility.

In continuous state spaces, a common approach in the MCTS literature is to use progressive widening, where the initial number of child nodes is limited and gradually increases as the node is visited more frequently [8]. This generates an adaptive search tree whose local resolution depends on the visit count of each node. In practice, however, this implementation introduces additional hyperparameters and complexity. In this project, we adopt a simpler approach by discretizing the continuous state space into a fixed finite number of bins. This approach provides a computationally efficient approximation that performs well for the purposes of this project.

III. PROBLEM FORMULATION

Secret Hitler is inherently a multi-agent game that would lend itself well to being formulated as a Partially Observable Stochastic Game (POSG). However, in order to make it tractable to solve we formulated the problem into a single agent MDP with simplified game mechanics. The primary simplifications and assumptions made included:

- Assuming all other agents played a fixed greedy policy that was embedded into the environment - based on the work by [3] and [1].
- The discard mechanics of the deck was simplified so that any discarded cards during the president and chancellor action phases would go directly back into a shuffled deck. This avoided need to both track the discarded cards explicitly and to re-shuffle mid-game.
- We only play the game from the fascist players perspective to make the problem an MDP as the fascist player knows all of the true roles of the other players (based on [1]). If the game was played from the liberal role it would necessitate being a POMDP and be subsequently much harder to solve as found in [3].
- To limit our state space we restricted the game to the minimum number of players (five).
- Detailed voting mechanics for the chancellors role were removed and replaced with a proxy based on the relative suspicion levels of each player being a fascist. This was an extension of the work from [1] and [2].
- Special game modifiers were removed so that the action space was limited to only three discrete choices (again to make the solution space tractable).

For the five player version of this game, there is one fascist (the agent we play as), Hitler (the other fascist) and three liberal players. The MDP was represented by $M = \{\gamma, S, S_T, A, T, R\}$ where:

- $\gamma := 0.99$ (a high discount factor was chosen because emphasis is placed on the future reward of winning the game

rather intermediate rewards.)

- $S := (n_{FP}, n_{LP}, x_s, x_r, i_c, i_p)$, where S represents the space of all possible game states and:

- $n_{FP} :=$ The number of fascist cards that have been played and $n_{FP} \in \{0, \dots, 6\}$.
- $n_{LP} :=$ The number of liberal cards that have been played and $n_{LP} \in \{0, \dots, 5\}$.
- $x_s :=$ A vector of probabilities that each player is fascist, where $x_s \in [0, 1]^5$ and the probability for each player is determined collectively by all players in the game.
- $x_r :=$ A vector containing the roles of all players where $x_r \in Roles^5$ and $Roles = \{Liberal, Fascist, Hitler\}$. $x_r[1]$ was always designated to be the real agent playing the game as a fascist.
- $i_c :=$ The player index corresponding to the current chancellor, where $i_c \in \{1, \dots, 5\}$.
- $i_p :=$ The player index corresponding to the current president, where $i_p \in \{1, \dots, 5\}$.

- $S_T :=$ The terminal states for the game were any $s \in S$, such that either $n_{FP} = 6$ (fascists win) or $n_{LP} = 5$ (liberals win).

- A : The action space for the MDP varied as a function of the game state and represented how the real player (player 1) interacted with the game.

- If $i_c = 1 || i_p == 1 \implies A = \{\text{Discard Fascist Card (DF)}, \text{Discard Liberal Card (DL)}\}$. Player 1 is in a position of power and can choose what cards to discard.
- Otherwise $\implies A = \{\text{No Action (NA)}\}$. No action can be taken as player 1 is not in a position of power.

- $R :=$ The model presented below was chosen by modifying the version of the reward function presented in [1]. It was modified to use potential based reward shaping so that optimal policies for the reward function are also the optimal policies for winning the game.

$$R(s, s', a) = \begin{cases} 1500 + F(s, s'), & n_{FP} = 6 \\ -1500 + F(s, s'), & n_{LP} = 5 \\ F(s, s'), & \text{otherwise} \end{cases} \quad (1)$$

$$F(s, s') = \begin{cases} -\phi(s), & s \in S_T \\ \gamma\phi(s') - \phi(s), & \text{otherwise} \end{cases} \quad (2)$$

$$\phi(s) = 3^{n_{FP}} - e^{n_{LP}} \quad (3)$$

The reward function $R(s, s', a)$ above was only used if the action provided to the generative model was valid (see Figure 1). The reward and state transition for an invalid action were:

$$(i_c = 1 || i_p = 1) \ \& \ a = \text{No Action} \implies r = -1, s' = s \quad (4)$$

$$(i_c \neq 1 \ \& \ i_p \neq 1) \ \& \ a \neq \text{No Action} \implies r = -1, s' = s \quad (5)$$

- $T :=$ The transition model used for the MDP was defined implicitly using a generative model due to the complexities

associated with it. Figure 1 shows the simplified game model used in this project.

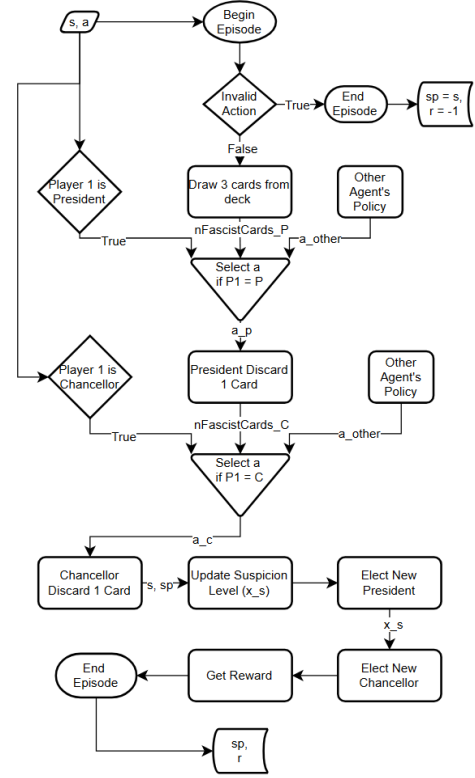


Fig. 1. Generative Model Functional Flow Diagram

The deck of cards from which the president draws from starts with 11 fascist policies and 6 liberal policies which are depleted as the game progresses and chancellors play cards. In each round, three cards are drawn from the desk and passed to the president. If the president is player 1, then the learned policy will be used to choose to discard one of these cards, otherwise the other agent's policy is used. The chancellor is passed the remaining two cards and then also discards a single card according to the current chancellors policy (either real player or other agent). The card that is left after this process deterministically updates either n_{FP} or n_{LP} of s' depending on if it is liberal or fascist.

The suspicion level ($x_s \in s'$) of the president and chancellor is then updated using Bayes rule based on the work presented in [2]. Given the policy that was played in that round, the new suspicion levels are:

$$P(C = F | Pol) = \frac{P(Pol | C = F)P(C = F)}{\sum_{c \in Roles} P(Pol | C = c)P(C = c)} \quad (6)$$

$$P(Pr = F | Pol) = \frac{P(Pol | Pr = F)P(Pr = F)}{\sum_{pr \in Roles} P(Pol | Pr = pr)P(Pr = pr)} \quad (7)$$

Where C = Chancellor's role, Pol = Policy played by the chancellor (Liberal/Fascist), Pr = President's role and F = Fascist and L = Liberal. Note that Hitler in this Bayes update has the same effect as a Fascist player. By marginalizing the

distribution, the opposite player of power's role can then be included:

$$P(\text{Pol}|C = c) = \sum_{pr \in \text{Roles}} P(\text{Pol}|C = c, Pr = pr) \quad (8)$$

$$P(\text{Pol}|Pr = pr) = \sum_{c \in \text{Roles}} P(\text{Pol}|C = c, Pr = pr) \quad (9)$$

Then, assuming all players play deterministically with a greedy policy, the probabilities of policies being played given the chancellor's and president's roles are:

$$P(\text{Pol} = F|C = F, Pr = F) = 1 - \frac{\binom{nL}{3}}{\binom{nL+nF}{3}} = 1 - P(\text{Pol} = L|C = F, Pr = F) \quad (10)$$

$$P(\text{Pol} = F|C = L, Pr = L) = \frac{\binom{nF}{3}}{\binom{nL+nF}{3}} = 1 - P(\text{Pol} = L|C = L, Pr = L) \quad (11)$$

$$P(\text{Pol} = F|C = F, Pr = L) = \frac{\binom{nF}{2}\binom{nL}{1}}{\binom{nL+nF}{3}} + \frac{\binom{nF}{3}}{\binom{nL+nF}{3}} \quad (12)$$

$$\begin{aligned} P(\text{Pol} = F|C = F, Pr = L) &= \\ P(\text{Pol} = F|C = L, Pr = F) &= \\ 1 - P(\text{Pol} = L|C = F, Pr = L) &= \\ 1 - P(\text{Pol} = L|C = L, Pr = F) & \end{aligned} \quad (13)$$

Where nL and nF are the number of remaining liberal and fascist cards in the deck at the start of the round.

The final phase in a round involves selecting the next president and chancellor. The president's role deterministically transitions to the next player ($i_p \leftarrow i_p + 1$) (with wrapping as a circular buffer). The next chancellor is selected stochastically as a distribution of x_s such that:

$$P(i'_c = i) = \frac{e^{\lambda(1-x'_s)^i}}{\sum_{i=1}^5 e^{\lambda(1-x'_s)^i}} \quad (14)$$

$s.t. P(i'_c = i) = 0, \text{ if } i = i_c \parallel i = i'_p$

The exponential distribution on the trustworthiness of the players was used based on the work by [2] and influenced by the softmax function. λ values of 1, 5, 10, 50, and 150 were tested to capture different levels of risk adversity, with larger lambda resulting in a stronger preference to pick players that had a low suspicion level for the chancellor role, and smaller lambda resulting in a more random uniform selection.

Finally, when the environment was reset after the terminal state was reached, the player roles (except player 1), president's role and chancellor's role were all randomized and the suspicion levels set uniformly to 0.4 (e.g a uniform prior on fascist suspicion level given 2/5 players are known to be fascist/Hitler).

IV. SOLUTION APPROACH

A. Heuristic Policies

To evaluate the performance of the Deep Q-Network and Monte Carlo Tree Search solvers, two heuristic policies were constructed. The first heuristic policy is a "Random" policy that selects a random action between "Discard Liberal Card" and "Discard Fascist Card" when in a position of power. The second heuristic policy is a "Greedy" policy that always plays the "Discard Liberal Card" action when in power, meaning it

always wants to enact a fascist policy and does not care about raising its own suspicion level. Both of these policies only play valid actions, so they only play a discard action if they are in a position of power and play "No Action" otherwise.

B. Deep Q-Network Solver

The DQN solver approximates the optimal state-action value function $Q^*(s, a)$ with a neural network as described in [4]. The optimal state-action value function is given by the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim T(\cdot|s, a)} \left[R(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \quad (15)$$

The DQN algorithm trains the Q-network by interacting with the environment via an off-policy exploration algorithm and storing experience tuples in a data buffer. Mini-batches are then randomly sampled from the buffer and used to calculate and minimize the temporal-difference loss between predicted Q-values and Q-values from a target network. The pseudocode for the algorithm as applied to Secret Hitler is shown in Algorithm 1, and the implementation details follow.

Algorithm 1 Variable Length Multi-Step Learning DQN

```

1: for step = 1 to max_steps do
2:    $a \leftarrow \epsilon\text{-greedy}(Q, s)$  with action masking
3:    $s', r \leftarrow G(s, a)$ 
4:   if  $a \neq \text{NO\_ACTION}$  then
5:     if pending exists, put  $(s_p, a_p, R_p, s, \text{false}, k_p)$  in  $\mathcal{B}$ 
6:      $(s_p, a_p, R_p, k_p) \leftarrow (s, a, r, 1)$ 
7:   else if pending exists then
8:      $R_p += \gamma^{k_p} \cdot r$ ;  $k_p += 1$ 
9:   end if
10:  if terminal( $s'$ ) then
11:    if pending exists, put  $(s_p, a_p, R_p, s', \text{true}, k_p)$  in  $\mathcal{B}$ 
12:    clear pending;  $s \leftarrow \text{INITIALSTATE}()$ 
13:  else  $s \leftarrow s'$ 
14:  end if
15:  if  $|\mathcal{B}| \geq \text{batch\_size}$  and step > warmup then
16:    for  $i = 1$  to updates_per_step do
17:      sample batch  $\mathcal{D}$  from  $\mathcal{B}$ 
18:      for each  $(s, a, R, s', d, k) \in \mathcal{D}$  do
19:        if  $d$  then  $y \leftarrow R$ 
20:        else  $y \leftarrow R + \gamma^k \max_{a'} [Q_{\text{tgt}}(s') + \text{mask}(s')]$ 
21:        end if
22:         $\ell \leftarrow (y - Q(s)[a])^2$ 
23:      end for
24:       $\mathcal{L} \leftarrow \frac{1}{|\mathcal{D}|} \sum \ell$ 
25:      update  $Q$  by gradient descent on  $\mathcal{L}$ 
26:    end for
27:  end if
28:  if step mod target_freq = 0 then  $Q_{\text{tgt}} \leftarrow Q$ 
29:  end if
30:  if step mod eval_freq = 0 then
31:     $w \leftarrow \text{evaluate}(Q)$ 
32:    if  $w > \text{best\_score}$  then  $Q^* \leftarrow Q$ ,  $\text{best\_score} \leftarrow w$ 
33:  end if
34:  decay  $\epsilon$ 
35: end for
36: return  $Q^*$ 

```

To approximate $Q^*(s, a)$ for the Secret Hitler problem, the state s must be encoded into a numerical vector. The number of fascist and liberal cards played, and the fascist suspicion levels of every player are directly included as continuous features, and one-hot encodings are used for the roles of all players

and the current chancellor and president. The full encoding is described in Table I, yielding a 32-dimensional input vector. The network features a single hidden layer of 128 neurons with a ReLU activation function and a final layer of size 3 corresponding to the action space.

TABLE I
STATE VECTOR ENCODING

Component	Encoding	Indices	Length
Fascist Policies Played	Scalar Count	1	1
Liberal Policies Played	Scalar Count	2	1
Fascist Suspicion Levels	Vector in $[0, 1]^5$	3-7	5
Player Roles	One-Hot per Player [F, L, H]	8-22	15
Current President	One-Hot Over 5 Players	23-27	5
Current Chancellor	One-Hot Over 5 Players	28-32	5

To explore the environment, an ϵ -greedy algorithm is utilized, which chooses a random action with probability ϵ ; otherwise, it selects the optimal action according to the Q-network. As this environment involves illegal actions in certain states, the exploration algorithm considers this, never taking an illegal action and only randomizing or optimizing over legal actions. Additionally, as the network trains, ϵ decays linearly from 0.5 to 0.05.

While standard DQN stores an experience tuple after every step in the environment, this performs poorly in the Secret Hitler environment because our agent only makes meaningful decisions when holding a position of power (president or chancellor), and there is a variable number of rounds between when the agent holds power. This causes issues with credit assignment when the state transitions independently of the agent's actions. To ensure that learning occurs through meaningful experience, we employ multi-step learning as described in [5], in which the sum of discounted rewards is accumulated over a series of steps rather than just the immediate reward. While most multi-step learning algorithms deploy this technique with a fixed horizon to accumulate rewards, we accumulate variably. Rewards begin accumulating when the agent takes an active action (Discard Liberal or Discard Fascist), and stop accumulating at the next point where the agent takes an active action or when the game ends. This can equivalently be viewed as a reformulation of the problem as a Semi-Markov Decision Process (SMDP), where actions take variable amounts of time to complete [6]. Thus, the experience tuple stored in the data buffer becomes:

$$\left(s_t, a_t, \sum_{i=0}^{k-1} \gamma^i r_{t+i}, s_{t+k}, done, k \right) \quad (16)$$

where s_t and a_t are the state and action at the agent's decision point, k is the number of environment steps until the next decision point or the end of the game, $\sum_{i=0}^{k-1} \gamma^i r_{t+i}$ is the discounted sum of rewards accumulated over those k steps, s_{t+k} is the state at the next decision point, and $done$ is a binary flag indicating whether the game has terminated.

In addition to multi-step learning, an action mask is applied to the Q-values of target network Q_{tgt} such that all illegal

actions are set to $-\infty$ as in [7]. With this mask and multi-step learning, the loss function becomes:

$$\mathcal{L}(\theta) = (y - Q_\theta(s, a))^2 \quad (17)$$

where the target y is defined as

$$y = \begin{cases} R & \text{if terminal} \\ R + \gamma^k \max_{a' \in \mathcal{A}_v(s')} Q_{tgt}(s', a') & \text{otherwise} \end{cases} \quad (18)$$

To improve stability, the loss is minimized over a batch of data, rather than a single experience tuple. During training, a batch of 40 samples is pulled from the data buffer at a time, and the average loss is minimized with an Adam Optimizer with a learning rate of 5×10^{-4} . The Adam optimizer performs four gradient steps for every step that the algorithm takes in the environment after a warm-up of steps in the environment.

To provide a stable target network Q_{tgt} , the target network is set to the trained network Q_θ every 1000 steps in the environment. This allows the network to approximate a moving target without instability caused by gradient updates to the target network. Lastly, as Q-value estimates fluctuate greatly during training, the trained network is evaluated every 2000 steps in the environment. The network is evaluated by computing the win rate and average reward across 10,000 Monte Carlo simulations, and the best network is saved and returned after the training process. The model was trained across 500,000 environment steps, with the best-performing network being returned for evaluation. After training, the learned policy simply selects the action that maximizes the learned Q-function for a given state, and was tested with evaluation on 100,000 simulations.

C. Monte Carlo Tree Search Solver

We also used MCTS as an online method for action selection. Whenever an agent must act, the algorithm builds a new search tree of depth d from the current game state. It iteratively performs simulated rollouts through the game, and stores visit counts $N(s, a)$, transition counts $T(s, a, s')$, and estimates for the state-action value $Q(s, a)$. Exploration action selection is governed by the Upper Confidence Bound (UCB1):

$$a = \arg \max_{a \in A} \left(Q(s, a) + c \sqrt{\frac{\log N(s)}{N(s, a)}} \right)$$

where, c is an exploration parameter.

We chose an exploration constant of $c = 5000$, a search depth of $d = 5$ and a maximum of 500 MCTS iterations per move. The choice of c was guided by the bounds of the value function, using the heuristic

$$c = 2(\bar{V} - \underline{V})$$

where $\bar{V}$ and $\underline{V}$ are the upper and lower bounds of the value function V . The exact details of this calculation are given in Appendix A. The depth and number of MCTS iterations were selected by trial and error to maximize the win rate of the MCTS solution.

A big challenge to this problem is the presence of a continuous belief variable (i.e. the suspicion level), making our current formulation not immediately applicable to MCTS. To make this tractable, we discretize the state-space by multiplying the suspicion level by 10^k and rounding to the nearest integer. We chose $k = 5$ as the number of digits, which generates a large but finite state space. This was acceptable, as there was only a finite number of deterministic s' available from a given s , due to the finite number of cards being drawn per round and a Bayesian Suspicion Update. Hence, by using this discretization approach continuous states could be represented in the MCTS tree without any loss of accuracy (see Appendix B for details).

Because the search tree only explores until a given depth d , we evaluate leaf-nodes through Monte Carlo rollouts. When a leaf node is reached, the algorithm calls the rollout function $U_{est}(s_{leaf})$. This function simulates the rest of the game from the leaf node to a terminal state using a greedy heuristic policy for all players. A greedy policy was used for the rollouts based on the near optimality observed in the DQN work. In each step of the rollout, actions are chosen according to this heuristic, and the next state and reward are sampled from the generative model. This is used to calculate an estimate of the state action value function at the leaf node $q(s_{leaf}, a)$ by computing

$$q_{est} = \sum_{t=0}^{T-1} \gamma^t R(s_t, s_{t+1}, a)$$

This estimated value function is then propagated backwards, updating the visit counts $N(s, a)$ and updates the average state action value function $Q(s, a)$ by

$$Q(s, a) \leftarrow Q(s, a) + \frac{q_{est} - Q(s, a)}{N(s, a)}$$

Over the course of 500 simulations, we return the action evaluated from the root of the tree:

$$a^* = \arg \max_a Q(s, a)$$

This corresponds to the online policy, where agents compute the approximate optimal actions at their current decision point.

V. RESULTS

To measure the performance of the heuristic policies and the trained DQN and MCTS solvers, 100,000 games were simulated for λ values of 1, 5, 10, 50, and 150, which represent varying degrees of concentration in the chancellor election probabilities given the suspicion levels, with $\lambda = 1$ being the most uniform (least risk averse) and $\lambda = 150$ being the most deterministic (most risk averse), see Section III for details. For each 100,000 game simulation, the policy's win rate and average return were calculated and are shown in Table II.

Based on these results, it is clear that the Greedy Heuristic, DQN, and MCTS outperform the Random Heuristic greatly across all λ values. Among the Greedy Heuristic, DQN, and MCTS policies, the Greedy Policy marginally performs the

best for the $\lambda = 1$ case, and DQN/MCTS performs best for all other λ values, with an increasing performance gap as λ increases. The mean MCTS win rate appears to outperform DQN for $\lambda = 50, 150$, however, this result is not statistically significant using a two-sample Student's-t test. This indicates that while the DQN and MCTS solvers likely have a near-greedy policy for lower levels of lambda, they exploit the per-player suspicion features at higher λ . It is also of note that the win rate of all policies tends to decrease with increasing λ , showing that more deterministic chancellor selection makes it harder for fascists to be elected into power and win the game.

Additionally, there are no instances where one policy has a higher win rate but lower average return, or vice versa. This indicates that the potential-based reward function aligns with the win-rate objective and policies optimal for the reward function are also optimal for winning.

To evaluate the training progress of DQN, the learning curve, loss curve, and win rate curve are shown in Figures 2-4 for the training process of $\lambda = 150$. Although fairly unstable, as is common for DQN training, the average return and win rate do show a positive correlation with the number of steps in the environment, and the loss function tends to decrease with experience.

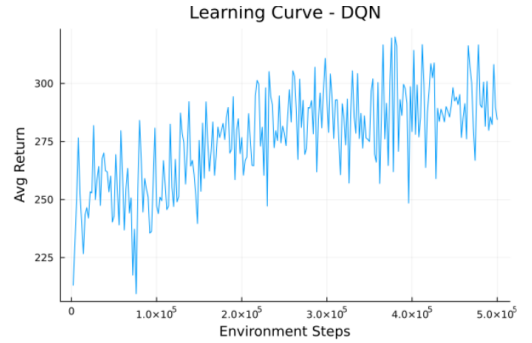


Fig. 2. Avg. Return of DQN during training ($\lambda = 150$)

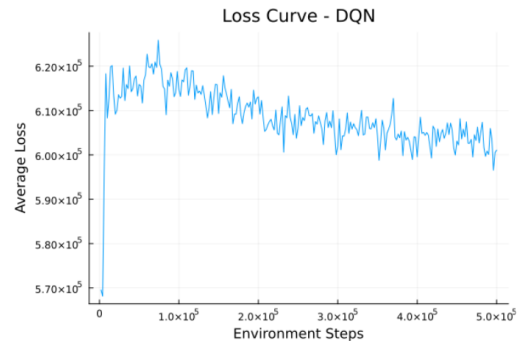


Fig. 3. Avg. Loss of DQN during training ($\lambda = 150$)

Figure 5 shows two layers of an MCTS tree ran from a starting state where player 1 is the chancellor. In this example, the value of taking the Discard Fascist action (467.9) is almost as high as the Discard Liberal action (528.9). Without the

TABLE II
POLICY PERFORMANCE ACROSS λ VALUES (100,000 GAMES EACH)

Policy	Metric	$\lambda=1$	$\lambda=5$	$\lambda=10$	$\lambda=50$	$\lambda=150$
Greedy	Win Rate	0.782 $\pm$ 0.001	0.664 $\pm$ 0.001	0.618 $\pm$ 0.002	0.596 $\pm$ 0.002	0.593 $\pm$ 0.002
	Avg Return	781.6 $\pm$ 3.6	454.0 $\pm$ 4.1	327.0 $\pm$ 4.3	265.9 $\pm$ 4.3	256.5 $\pm$ 4.3
DQN	Win Rate	0.779 $\pm$ 0.001	0.668 $\pm$ 0.001	0.624 $\pm$ 0.002	0.601 $\pm$ 0.002	0.605 $\pm$ 0.002
	Avg Return	772.3 $\pm$ 3.6	465.9 $\pm$ 4.1	341.1 $\pm$ 4.2	277.5 $\pm$ 4.3	290.2 $\pm$ 4.3
MCTS	Win Rate	0.773 $\pm$ 0.001	0.667 $\pm$ 0.001	0.624 $\pm$ 0.001	0.606 $\pm$ 0.002	0.607 $\pm$ 0.002
	Avg Return	755.9 $\pm$ 3.6	463.6 $\pm$ 4.1	341.2 $\pm$ 4.2	294.6 $\pm$ 4.3	295.9 $\pm$ 4.3
Random	Win Rate	0.578 $\pm$ 0.002	0.488 $\pm$ 0.002	0.455 $\pm$ 0.002	0.440 $\pm$ 0.002	0.438 $\pm$ 0.002
	Avg Return	212.5 $\pm$ 4.3	-37.2 $\pm$ 4.4	-128.6 $\pm$ 4.4	-171.1 $\pm$ 4.3	-175.2 $\pm$ 4.3

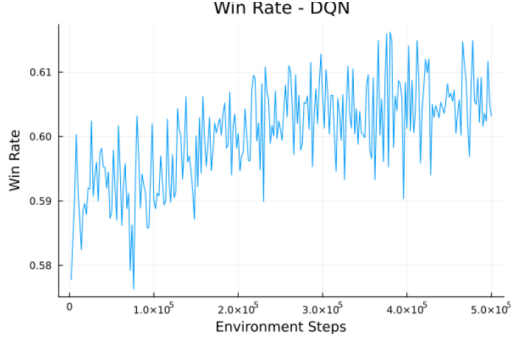


Fig. 4. Win Rate of DQN during training ($\lambda = 150$)

exploration bonus, this relative similarity in $Q(s,a)$ values would not occur due to the higher reward of discarding a liberal card. You can also observe the effect of the action masking, where the No Action (NA) action is never visited due to its large negative Q value. When running the MCTS solver, each game took on average 0.25 seconds indicating that it was capable of running real time if playing the real Secret Hitler board game.

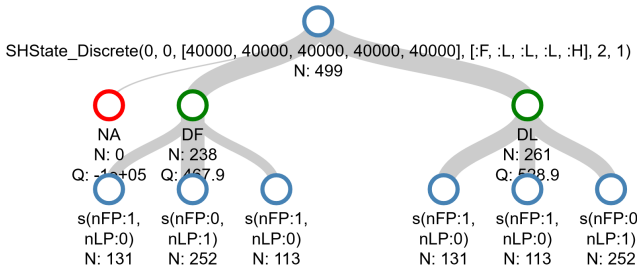


Fig. 5. MCTS Example Tree With Simplified State Descriptions (nFP = number of fascist policies played, nLP = number of liberal policies played).

To analyze the DQN and MCTS policies for the different λ values, the number of Discard Liberal (DL) and the number of Discard Fascist (DF) actions were recorded for 100 games of each λ value, and are shown in Table III. Here, we can see that for both solvers, the number of Discard Fascist actions tends to increase with increasing λ , showing that the agent learns to lower its suspicion level more when chancellor elections are

more deterministic. It is also interesting to note that the total number of actions (excluding No Action) taken across λ values tends to decrease as λ increases, illustrating the difficulty of getting into positions of power when λ is high.

TABLE III
ACTION COUNTS FOR DQN AND MCTS ACROSS λ VALUES (100 GAMES EACH)

	DQN			MCTS		
	DF	DL	$\frac{DF}{DL} \%$	DF	DL	$\frac{DF}{DL} \%$
$\lambda = 1$	2	352	0.57	23	321	7.17
$\lambda = 5$	23	292	7.88	43	269	15.99
$\lambda = 10$	38	255	14.90	31	279	11.11
$\lambda = 50$	53	257	20.62	42	257	16.34
$\lambda = 150$	48	259	18.53	48	252	19.05

VI. DISCUSSION AND CONCLUSIONS

1) *Analysis of Results:* Examining the results, both the DQN and MCTS agents clearly outperform the random action, indicating that the agents' actions do impact the fascists' win rate in Secret Hitler. However, DQN and MCTS only match or marginally exceed the greedy heuristic policy of only discarding liberal policies, with the gap widening at higher values of λ . This indicates that the greedy policy is near-optimal for a fascist agent in most game states, making the win rate improvement exhibited by the DQN or MCTS policies fairly small.

While the gap is still fairly small, DQN and MCTS do have a larger performance gap at higher λ . This makes sense because at lower values of λ , chancellor selection is less dependent on suspicion, so there is less incentive for an agent to lower its suspicion level. Thus, at higher values of λ , the DQN and MCTS solvers learn to play discard fascist actions more to lower their suspicion level, leading to better performance over a greedy policy that never attempts to do so.

While DQN and MCTS take very different approaches, as an offline value learning algorithm versus an online tree search, they both converge on similar strategies, suggesting that both are finding a similar near-optimal policy. This indicates that both solution approaches work well, and that only observing

marginal improvements over a greedy policy is a result of the greedy policy being nearly optimal for our implementation of the game.

2) *Departure from Initial Proposal:* In this project we achieved all of our goals from the minimum working example and our main approach levels. However, we did depart from the project proposal in formulating the problem as an MDP, rather than the originally proposed POMDP. We made the choice to switch from a POMDP to a MDP for the following reasons.

When we started developing our model it became apparent that the fascist role did not lend itself as well to a POMDP (as the fascist player new the roles of all other players by default). The liberal player role was better suited to a POMDP formulation, but was not as interesting in terms of trying to find an optimal policy, as liberal players have no incentive to not play liberal cards.

At the time of formulating the project proposal we also had limited exposure to solving POMDP's and did not fully realize how difficult they can be to solve. Therefore, whilst the fascist role could be formulated into a POMDP by making the suspicion levels of each player not fully observable, we felt that it would be more valuable to avoid this extra complexity. Hence, moving to an MDP allowed us to focus on implementing skills we learned on a more tractable problem.

3) *Future Work:* If we were to repeat or extend upon this project we would consider the following improvements.

Firstly, we would try to implement the solver as a multi-agent game rather than a single agent problem. Whilst the difficulty to solve the problem would increase drastically, a lot of the game nuance and realism is lost by assuming the policies of other players.

Secondly, the Bayes suspicion update utilized in this work was limited and neglected a lot of the true game mechanics, including deception of what cards were drawn. Moving to a POMDP where the real agent held beliefs over suspicion levels based on information provided by the president and chancellor (which could include lies) would be a better model, and result in more applicable/interesting solutions.

Finally, by including some of the advanced game mechanics (E.g. assassinations or instant win conditions), and including proper voting processes, more realistic optimum policies are expected to be found. As compared to the relatively trivial policies identified in this project.

4) *Conclusions:* Overall, we successfully implemented DQN and MCTS to find near-optimal policies for our simplified MDP version of Secret Hitler. In solving this problem we demonstrated that you can achieve better performance than a greedy policy through using either the offline DQN or online MCTS solution, albeit the performance differences were marginal in terms of both reward and win rate. It was also found that the DQN and MCTS policies are only advantageous (relative to the greedy policy) when the group's collective suspicion of each player being a fascist is weighted heavily in the voting process (E.g. $\lambda \geq 10$, representing very distrusting players). Furthermore, MCTS and DQN arrive at similar

policies, based on their win rates and average rewards being within two standard deviations of each other for $\lambda \geq 5$. Finally, whilst our model of Secret Hitler was highly simplified, this project demonstrated in principle how a fascist agent could play in a highly suspicious environment in order to maximize their chance of winning the game.

VII. CONTRIBUTIONS AND ALGORITHMS

The work within this project was broken down as follows:

Adam: Responsible for model generation and help with MCTS implementation.

Shaan: Responsible for DQN implementation and training and Bayesian update.

Pedro: Responsible for MCTS implementation and testing.

In this project both the MCTS and DQN algorithms were developed by our team, building upon our homework solutions.

VIII. RELEASE STATEMENT

The authors grant permission for this report to be posted publicly.

IX. CODE

The code for this project can be access here:

<https://github.com/adam-yarrow/ASEN5264-Project>

REFERENCES

- [1] J. Zhang, and A. Peng, "Modeling the Board Game Secret Hitler as a Decision Making Problem", Unpublished Work, Stanford CS238, 2023
- [2] M. Aitchison, L. Benke, and P. Sweetser, "Learning to deceive in multi-agent hidden role games," in *Proc. DeceptAI: Int. Workshop on Deceptive AI*, 2022.
- [3] J. Reinhardt, "Competing in a complex hidden role game with information set Monte Carlo tree search," *arXiv preprint arXiv:2005.07156*, May 2020.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, Dec. 2013.
- [5] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. 32nd AAAI Conf. on Artificial Intelligence (AAAI-18)*, 2018, pp. 3215–3222.
- [6] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, Aug. 1999.
- [7] A. M. Ali and L. Tirlé, "Action masked deep reinforcement learning for controlling industrial assembly lines," in *Proc. 2023 IEEE World AI IoT Congress (AllIoT)*, 2023, pp. 0797–0803. DOI: 10.1109/AI-IoT58121.2023.10174426.
- [8] A. Couëtoux and H. Dohmen, "Adding Double Progressive Widening to Upper Confidence Trees to Cope with Uncertainty in Planning Problems," in *Proceedings of the 9th European Workshop on Reinforcement Learning (EWRL-9)*, Athens, Greece, Sep. 2011.

APPENDIX

A. MCTS Upper Confidence Bound Exploration Constant Estimate

The value function $V^\pi(s)$ is given by

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t R(s_t, s_{t+1}, a_t) | s_0 = s \right]$$

The reward function given in Eqs. (1-3) is expressed as the sum of the reward due to winning/losing and the reward shaping term $R(s, s', a) = R_{win}(s, s', a) + F(s, s')$. The value function becomes

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t R_{win}(s, s') \right] + \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t F(s, s') \right]$$

Applying Eq. 3 to the last term gives

$$\begin{aligned} \sum_{t=0}^{T-1} \gamma^t F(s_t, s_{t+1}) &= \sum_{t=0}^{T-1} \gamma^t (\gamma \phi(s_{t+1}) - \phi(s_t)) \\ &= \sum_{t=0}^{T-1} (\gamma^{t+1} \phi(s_{t+1}) - \gamma^t \phi(s_t)) \end{aligned}$$

Expanding the sum,

$$\begin{aligned} &= (\gamma \phi(s_1) - \phi(s_0)) + (\gamma^2 \phi(s_2) - \gamma \phi(s_1)) \\ &\quad + (\gamma^3 \phi(s_3) - \gamma^2 \phi(s_2)) \\ &\quad + \dots + (\gamma^T \phi(s_T) - \gamma^{T-1} \phi(s_{T-1})) \end{aligned}$$

Notice that all the middle terms cancel out, leaving only the first and last terms

$$\sum_{t=0}^{T-1} \gamma^t F(s_t, s_{t+1}) = -\phi(s_0) + \gamma^T \phi(s_T).$$

, and the value function simplifies to

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t R_{win}(s, s') \right] + \mathbb{E}_\pi [-\phi(s_0) + \gamma^T \phi(s_T)]$$

Now, let us now focus on the bounds of the value function $\bar{V}^\pi(s)$. The maximum of the first term is +1500 in the case where the player wins at $t = 0$. In the second term, since $\phi(s) = 3^{n_{FP}} - e^{n_{LP}}$, $|\phi(s)|$ is bounded by the number of liberal and fascist policies. With $n_{FP} \leq 6$ and $n_{LP} \leq 5$, we have $|\phi(s)| \leq 3^6 + e^5 \approx 1000$. Hence the upper and lower bounds are

$$\bar{V}^\pi(s) = 1500 + \max_s |\phi(s)| \approx 2500$$

$$\underline{V}^\pi(s) = -1500 - \max_s |\phi(s)| \approx -2500$$

Finally, we determine an initial exploration parameter c by the heuristic

$$c = 2(\bar{V} - \underline{V}) \approx 10000$$

We can also estimate the upper and lower bound on the value function numerically by assuming either the game results in all six fascists cards being played sequentially, or all five liberal cards being played sequentially, such that in either case the maximum or minimum possible reward is achieved at each time step. Using the reward function from Section III we get the rewards found in Table IV.

t	Rewards for $\bar{V}$	Rewards for $\underline{V}$
0	1.98	-1.7
1	5.92	-4.61
2	17.74	-12.51
3	53.2	-33.98
4	159.6	-1592.34
5	1978	N/A

TABLE IV

UPPER AND LOWER BOUNDS ON REWARDS TO INFORM VALUE FUNCTION.

These two methods yield a starting estimates for c of approximately 7300 and 10000. From these estimates, different iterations were trialed to find a good c value for the problem at hand. We chose a value of $c = 5000$.

B. Discretization of Continuous Suspicion Level

As mentioned in Section III we discretized the continuous suspicion level variable (x_s) to make MCTS tractable. By doing this it made the states completely discrete which was critical for generating the $Q(s, a)$, $N(s, a, s')$ and $T(s, a, s')$ dictionaries in MCTS. The reason why this approach worked is because there existed a finite number of possible s' given a particular initial s . This was a result of the game mechanics, where the continuous suspicion level x_s was deterministic in the current discrete state and stochastic in the card draws. However, as the number of different card combinations you could obtain in a given round was fixed (for a given s) this meant the maximum number of s' was also fixed.

By converting the suspicion level to an integer this allowed us to assign state-action counts, value updates and transition counts to the correct key in the MCTS dictionaries (as if the key was a floating point number there is a risk of creating a new key for an identical state with different machine precision truncation error). A number of different integer precision levels were trialed with $k = 5$ selected as it showed no errors in the assignment of states to discrete dictionary keys when running 1000 iterations of MCTS.

To illustrate our implementation consider the following example where we have the following five initial game states with identical x_r, i_c, i_p :

$$s_1 = \{0, 0, [0.4, 0.4, 0.4, 0.4, 0.4], x_r, i_c, i_p\}$$

$$s_2 = \{0, 0, [0.12345, 0.4, 0.4, 0.4, 0.4], x_r, i_c, i_p\}$$

$$s_3 = \{0, 0, [0.123450000001, 0.4, 0.4, 0.4, 0.4], x_r, i_c, i_p\}$$

$$s_4 = \{0, 0, [0.1234, 0.4, 0.4, 0.4, 0.4], x_r, i_c, i_p\}$$

$$s_5 = \{0, 0, [0.123456, 0.4, 0.4, 0.4, 0.4], x_r, i_c, i_p\}$$

Given the only difference between the states is the suspicion level of player 1, let us define the $N(s, a)$ dictionary in terms of the value of $x_s[1]$, then, assuming the same action (a) was

taken in MCTS for all of these states, without discretization we would obtain the following counts:

$$N(x_s[1] = 0.4, a) = 1$$

$$N(x_s[1] = 0.12345, a) = 1$$

$$N(x_s[1] = 0.123450000001, a) = 1$$

$$N(x_s[1] = 0.1234, a) = 1$$

$$N(x_s[1] = 0.123456, a) = 1$$

This is clearly not conducive to an effective MCTS algorithm, especially considering s_3 is equivalent to s_2 within floating point precision. Hence, by applying our discretization process with $k = 5$ we instead obtain:

$$N(x_s[1] = 40000, a) = 1$$

$$N(x_s[1] = 12345, a) = 3$$

$$N(x_s[1] = 12340, a) = 1$$

This result is much closer to the true counts if the work was done in the continuous space. The only discrepancy between the discretized and continuous solution in this case is that s_5 becomes associated with the s_2 and s_3 results when it is clearly a different state. However, this scenario was not observed to occur in our MCTS Monte-Carlo solutions for the discretization level (k) selected.